

KNOWLEDGE REPRESENTATION

Lecture 1: Introduction

dr Anna Maria Radzikowska

Warsaw University of Technology
Faculty of Mathematics and Information Science
Building MiNI PW, room 504

E-mail: A.Radzikowska@mini.pw.edu.pl

Warsaw 2026



Overview

- 1 Course regulations
- 2 Project checkpoints
- 3 Reasoning about Actions
- 4 Ontologies of actions
- 5 Main problems in RAC
 - Frame problem
 - Ramification problem
 - Qualification problem
- 6 Simple dynamic system
 - Action language $\mathcal{A}$
 - Statements in $\mathcal{A}$
 - Examples
 - Semantics of $\mathcal{A}$

Regulamin przedmiotu

- ❶ Przedmiot obejmuje: 45 godz. wykładu i 30 godz. zajęć projektowych.
- ❷ Przedmiot kończy się egzaminem. Student ma prawo przystąpić do wszystkich trzech terminów egzaminu (2 terminy w czerwcu, 1 termin we wrześniu).
- ❸ Uczestnictwo:
 - w wykładach nie jest obowiązkowa;
 - w zajęciach projektowych jest obowiązkowa w wyznaczonych punktach kontrolnych, w pozostałych terminach zajęcia odbywają się na życzenie studentów (dyskusje nad zadaniem, przedstawienie propozycji rozwiązań, wyjaśnienia niejasności itp.)
- ❹ Łączna ocena z przedmiotu obejmuje:
 - max. **20 punktów** z projektu (w sesji wrześniowej max. **15 punktów**);
 - max. **30 punktów** z egzaminu.
- ❺ Uzyskanie pozytywnej oceny z przedmiotu wymaga spełnienia następujących dwóch warunków: **min. 11 punktów** z części projektowej oraz **min. 16 punktów** z egzaminu.

⑥ Część projektowa:

- W trakcie semestru studenci piszą duży projekt zespołowy w grupach 7-8 osobowych. Zespół projektowy zostaje ustalony na początku semestru.
- Temat zadania jest losowany spośród kilku zaproponowanych przez prowadzącego. Nie może być on zmieniony w trakcie semestru.
- Projekt obejmuje: część teoretyczną, część implementacyjną i część testową.
- Każdy student uczestniczy we **wszystkich** trzech częściach projektu.
- Zespół projektowy wybiera **koordynatora**:
 - Odpowiada za sprawny przebieg prac.
 - Prezentuje poszczególne etapy projektu.
 - Jest aktywnie zaangażowany w każdym etapie prac projektowych.
 - Określa stopień zaangażowania członków zespołu, może przyznać do 2p bonusowych co najwyżej 2 osobom szczególnie zaangażowanym w przygotowanie projektu.

7 Część projektowa (cd.):

- Końcowe oddanie projektu obejmuje
 - ▷ Prezentację działania aplikacji.
 - ▷ Złożenie aplikacji (pliku wykonywalnego).
 - ▷ Złożenie (w formie papierowej i elektronicznej) następujących dokumentów:
 - opracowania teoretycznego,
 - dokumentacji technicznej,
 - instrukcji obsługi aplikacji,
 - opracowania przeprowadzonych testów,
 - dokładnego opisu udziału poszczególnych członków zespołu w przygotowaniu projektu z ich procentowym udziałem w tych pracach.

8 Część projektowa (cd.):

- Za projekt każdy otrzymuje max. 20p, jeśli projekt jest oddany terminowo (do 12.06.2025), jeśli projekt jest oddany w terminie wrześnieowym, każdy z członków zespołu może uzyskać max. 15p.
- Ocena indywidualna za projekt:
 - Każdy otrzymuje po max. **8p** za część teoretyczną i aplikację oraz max. **4p** za część testową.
 - Finalna ocena zależy od indywidualnego wkładu w te części, w których student uczestniczył, oraz od jakości tej części.
 - Ocena za część teoretyczną/aplikację:
 - Niech w tej części uczestniczy N osób. Wówczas max. punktacja: $P = N \cdot 8$ punktów. (np. dla $N = 8$, $P = 64p$). — Ocena za tę część $S\%$. Wówczas punktacja: $p = \frac{S}{100} \cdot P$ punktów (np. dla $S = 75\%$, $p = 48p$).
 - Student(ka) ma wkład $k\%$. Otrzymuje wtedy $pkt = \frac{k}{100} \cdot p$ punktów, lecz nie więcej niż 8p. (np. $k = \frac{1}{6} = 16,7\%$, $pkt = 8p$).
 - Ocena za część testową jest analogiczna, przy czym tu $P = N \cdot 2p$, gdzie N to liczność zespołu.

9 Egzamin:

- Egzamin obejmuje jedynie część pisemną i jest obowiązkowy dla wszystkich studentów.
- Składa się z części testowej (ocenianej na max. 20p) i zadaniowej (ocenianej na max. 10p.)
- Część testowa jest testem wielokrotnego wyboru. Obejmuje 10 pytań testowych, z których każde ma 4 opcje wyboru, przy czym dokładnie jedna, dwie, trzy lub cztery z nich mogą być prawidłowe. Poprawną odpowiedzią na pytanie jest zaznaczenie **wszystkich** opcji prawidłowych i **brak zaznaczenia** jakiegokolwiek nieprawidłowej. Podanie poprawnej odpowiedzi jest oceniane na **2p**, odpowiedź błędna jest oceniana na **0p**.
- Część zadaniowa obejmuje 1 zadanie, należy podać **dokładne** rozwiązanie i je **precyzyjnie** uzasadnić.

Regulamin przedmiotu (cd.)

- 10 Ocena ogólna z przedmiotu wyznaczana jest na podstawie uzyskanych punktów:

Przedział punktowy	Ocena
[27, 31]	dost
(31–36]	dost $\frac{1}{2}$
(36–41]	db
(41–46]	db $\frac{1}{2}$
powyżej 46	bdb

- 11 Materiały z wykładów oraz aktualne informacje są dostępne na MS Teams w zespole “RW 2026”. oraz pod adresem

[https://pages.mini.pw.edu.pl/~radzikowskaa/Courses/RW 2026/Lectures](https://pages.mini.pw.edu.pl/~radzikowskaa/Courses/RW%202026/Lectures)

Punkty kontrolne

Punkt kontrolny 1: Postęp w części teoretycznej

Wstępna prezentacja części teoretycznej projektu.

26.03.2026 (czwartek)

- Projekt 1: 15:30 – 15:45
- Projekt 2: 15:45 – 16:00
- Projekt 3: 16:15 – 16:30

27.03.2026 (piątek)

- Projekt 4: 15:15 – 15:30
- Projekt 5: 15:30 – 15:45

Punkt kontrolny 2: Oddanie części teoretycznej

Pełna prezentacja części teoretycznej projektu, w tym:

- wydrukowany opis (w LaTeXu!) części teoretycznej projektu,
- odpowiadający mu plik PDF na CD.

23.04.2026 (czwartek)

- Projekt 1: 14:15 – 15:00
- Projekt 2: 15:00 – 15:45
- Projekt 3: 16:00 – 16:45

24.04.2026 (piątek)

- Projekt 4: 14:15 – 15:00
- Projekt 5: 15:00 – 15:45

Punkt kontrolny 3: Postęp w aplikacji

Wstępna prezentacja aplikacji.

12.05.2026 (wtorek, jak piątek)

- Projekt 1: 15:15 – 15:30
- Projekt 2: 15:30 – 15:45

12.05.2026 (czwartek)

- Projekt 3: 15:30 – 15:45
- Projekt 4: 15:45 – 16:00
- Projekt 5: 16:15 – 16:30

Punkt kontrolny 4: Prezentacja aplikacji i złożenie projektu

Pełna prezentacja implementacji projektu. Należy złożyć:

- Dokumentację projektu (plik PDF przygotowany w systemie LaTeX) zawierającą:
 - opis części teoretycznej projektu,
 - opis dokumentacji technicznej,
 - instrukcję użytkowania aplikacji,
 - wykonane testy aplikacji,
 - szczegółowy opis udziału każdego z członków zespołu w poszczególnych częściach prac projektowych.
- Aplikację – **plik wykonywalny** w systemie Windows, nie może to być aplikacja konsolowa!

Punkt kontroly 4 (cd)

3.06.2026 (środa, jak piątek)

- Projekt 1: 14:15 – 15:00
- Projekt 2: 15:00 – 15:45

11.06.2026 (czwartek)

- Projekt 3: 14:15 – 15:00
- Projekt 4: 15:00 – 15:45
- Projekt 5: 16:15 – 16:45

- ① Reasoning about Actions and Change.
- ② Foundations to classical logic.
 - Automated theorem proving - Resolution method.
- ③ Models and types of knowledge.
 - Epistemic logic.
 - Dynamic and temporal logic.
 - BDI systems.
- ④ Default logic.
- ⑤ Rough Sets and learning from examples.

Reasoning about Actions

Reasoning about actions

Dynamic system

A **dynamic system** (DS) might be viewed as

- a collection of objects, together with their properties, and
- a collection of actions which, while performed, change properties of objects (in consequence, the state of the world).

Main task

One of the main tasks in modelling DS is the following:

Define reasoning methods that allow for deriving conclusions about necessary/possible results of performing actions.

Reasoning about Actions (RAC)

The above task is the main objective in **Reasoning about Actions (RAC)**.

Applications of RAC

- AI systems that operate in real words
- Design of autonomous agents (involving robotics, MAS)
- Planning tasks
- Model-based diagnosis
- Automatic control of complex systems
- ...

Effects of actions

- deterministic (only one outcome), non-deterministic (at least two different outcomes)
- certain, typical (preferred)
- direct, indirect
- occurring immediately after an action ends; delayed effects
- all (some) effects are known.

Ontologies of actions (cont.)

Course of actions

- 1-step actions, actions with durations
- sequential actions, concurrent actions
- hierarchical occurrences (actions/subactions), non-hierarchical occurrences.

Preconditions of actions

If do not hold, the effects of an action are unknown (the effects are empty).

Main problems in RAC

Frame problem (McCarthy & Hayes, 1969)

The difficulty is that of indicating and inferring all those things that do not change when actions are performed.

Ramification problem (Finger, 1987)

Concerns the problem of concisely representing indirect effects of actions (propagation of changes). It is usually unreasonable to explicitly enumerate all of the consequences of actions.

Qualification problem (McCarthy, 1977)

The number of preconditions of actions is usually immense and it is unreasonable (if ever possible) to explicitly enumerate and check all of (even most unlikely) possibilities.

Yale Shooting Problem

There is a shooter Bill and a turkey Fred. Initially Fred is alive and Bill has an unloaded gun. Bill can perform two actions: loading the gun (LOAD) and shooting the gun (SHOOT). Loading the gun makes the gun loaded, while shooting the gun makes it unloaded and, in addition, Fred is not alive anymore, provided that the gun was loaded.

Conclusions

Natural conclusions:

- after loading the gun it is loaded and Fred is still alive
- after shooting the gun it is unloaded and Fred is not alive.

However, in classical logic we cannot infer that after loading the gun Fred is still alive.

Ramification problem

Modification of YSP

There is a shooter Bill and a turkey Fred. Whenever Fred is walking, it is alive. Assume that the gun is loaded. Shooting the gun makes it unloaded and, if it was loaded before shooting, Fred is no longer alive.

Natural conclusion

After shooting the gun Fred is no longer alive and, in addition, it does not walk!

In other words, "*Fred is not walking*" is the indirect effect of the shooting action.

Qualification problem

Potato in the tailpipe problem, McCarthy 1977

We want to start a car. Natural preconditions to do that are:

- the key must be turned in the ignition,
- there must be gas in the tank,
- the battery must be connected,
- the wiring must be intact,
- ...

Also, there need not be a potato in the tailpipe!

Problem

It is hardly possible (or practical) to check all of unlikely qualifications each time we are interested in using the car.

Simple dynamic system

Simple dynamic system

Assumptions

- Inertia law.
- Complete information about effects of actions.
- Determinism.
- Only direct effects of action.
- Only sequential actions.
- All actions are executable in every state.
- If the precondition of an action does not hold, its effect is empty.

$\mathcal{A}$ language

In order to represent this class of DS we will use action languages of the class $\mathcal{A}$.

Basic notions

A **signature** is a pair $\Upsilon = (\mathcal{F}, \mathcal{Ac})$ where

- $\mathcal{F}$ is a set of *fluents*,
- $\mathcal{Ac}$ is a set of *actions*.

A **literal** is either a fluent $f \in \mathcal{F}$ or its negation $\neg f$. A set of literals is a **condition**.

Notation:

For a fluent f we write $\bar{f}$ to denote the literal corresponding to f , i.e., either f or $\neg f$.

Statements in $\mathcal{A}$

Value statement

$$\overline{f} \text{ after } A_1, \dots, A_n$$

where $f \in \mathcal{F}$ and $A_i \in \mathcal{Ac}$, $i = 1, \dots, n$.

INTUITIVELY:

$\overline{f}$ holds after performing the sequence $A_1, \dots, A_n$ of actions in the initial state.

ABBREVIATION:

If $n = 0$ we use the abbreviation

$$\text{initially } \overline{f}$$

intuitively read as: "In the initial state $\overline{f}$ holds".

Statements in $\mathcal{A}$ (cont.)

Effect statement

$A \text{ causes } \bar{f} \text{ if } \bar{g}_1, \dots, \bar{g}_k$

where $A \in \mathcal{A}c$ and $f, g_1, \dots, g_k \in \mathcal{F}$.

INTUITIVE MEANING

If the action A is performed in any state satisfying $\bar{g}_1, \dots, \bar{g}_k$, then in the resulting state $\bar{f}$ holds.

ABBREVIATION:

If $k = 0$, then this statement is abbreviated to $A \text{ causes } \bar{f}$

Action domain

Any non-empty set D of statements is called an **action domain**.

Yale Shooting Problem

Example 1.1

There is a shooter Bill and a turkey Fred. Initially Fred is alive and Bill has an unloaded gun. Bill can perform two actions: loading the gun (LOAD) and shooting the gun (SHOOT). Loading the gun makes the gun loaded, while shooting the gun makes it unloaded and, in addition, Fred is not alive anymore, provided that the gun was loaded.

REPRESENTATION IN $\mathcal{A}$:

***initially** $\neg$ loaded;*

***initially** alive;*

LOAD **causes** loaded;

SHOOT **causes** $\neg$ loaded;

SHOOT **causes** $\neg$ alive **if** loaded.

Example 1.2

There is a shooter Bill and a turkey Fred. Shooting the gun makes it unloaded and, in addition, Fred becomes dead, if the gun was loaded at the beginning of shooting. Initially Fred is alive. After shooting the gun Fred is dead.

REPRESENTATION IN $\mathcal{A}$:

***initially** alive;*

SHOOT ***causes*** $\neg$ loaded;

SHOOT ***causes*** $\neg$ alive ***if*** loaded;

$\neg$ alive ***after*** SHOOT.

Example 1.3

A car was left for three consecutive days. Then it was found stolen.

REPRESENTATION IN $\mathcal{A}$:

initially $\neg stolen$;

stolen ***after*** WAIT, WAIT, WAIT.

State

A **state** is any mapping $\sigma : \mathcal{F} \rightarrow \{0, 1\}$. Σ stands for the set of all states.

For any $f \in \mathcal{F}$, if $\sigma(f) = 1$, then we say that f **holds in** σ and write $\sigma \models f$. If $\sigma(f) = 0$, then f does not hold in σ , in symbols $\sigma \not\models f$, or equivalently, $\sigma \models \neg f$.

A condition $\{\bar{f}_1, \dots, \bar{f}_n\}$ holds in σ iff for every $i = 1, \dots, n$, $\sigma \models \bar{f}_i$.

Semantics of $\mathcal{A}$ – transition function

Transition function

A **transition function** is any mapping $\Psi : \mathcal{Ac} \times \Sigma \rightarrow \Sigma$.

For any $\sigma \in \Sigma$ and for every $A \in \mathcal{Ac}$, $\Psi(A, \sigma)$ is the state resulting from performing the action A in the state σ .

The generalization of this function is the mapping $\Psi^* : \mathcal{Ac}^* \times \Sigma \rightarrow \Sigma$ as follows:

- $\Psi^*(\epsilon, \sigma) = \sigma$.
- $\Psi^*((A_1, \dots, A_n), \sigma) = \Psi(A_n, \Psi^*((A_1, \dots, A_{n-1}), \sigma))$.

Semantics of $\mathcal{A}$ – Semantical structures

$\mathcal{L}$ – an action language of the class $\mathcal{A}$ over the signature $\Upsilon = (\mathcal{F}, \mathcal{Ac})$.

Structure

A **structure** for $\mathcal{L}$ is a pair $S = (\Psi, \sigma_0)$ where Ψ is a transition function and $\sigma_0 \in \Sigma$ is the initial state.

Satisfaction

Let $S = (\Psi, \sigma_0)$ be a structure for $\mathcal{L}$. A statement s is **true in S** , in symbols $S \models s$, whenever

- if s is of the form

$\bar{f}$ **after** $A_1, \dots, A_n$,

then $\Psi((A_1, \dots, A_n), \sigma_0) \models \bar{f}$

- if s is of the form

A **causes** $\bar{f}$ **if** $\bar{g}_1, \dots, \bar{g}_k$,

then for every $\sigma \in \Sigma$ such that $\sigma \models \bar{g}_i, i = 1, \dots, k$, $\Psi(A, \sigma) \models \bar{f}$

Semantics of $\mathcal{A}$ – Models

Model

Let D be an action domain in the language of the class $\mathcal{A}$ over a signature $\Upsilon = (\mathcal{F}, \mathcal{Ac})$. A structure $S = (\Psi, \sigma_0)$ is a **model of D** iff

(M1) for every $s \in D$, $S \models s$,

(M2) for every $A \in \mathcal{Ac}$, for every $f, g_1, \dots, g_k \in \mathcal{F}$, and for every $\sigma \in \Sigma$, if one of the following conditions holds:

(i) for every effect statement in D of the form

$$A \text{ causes } \bar{f} \text{ if } \bar{g}_1, \dots, \bar{g}_k,$$

$$\sigma \not\models \bar{g}_i \text{ for some } i = 1, \dots, k,$$

(ii) D does not contain an effect statement

$$A \text{ causes } \bar{f} \text{ if } \bar{g}_1, \dots, \bar{g}_k$$

then $\sigma \models f$ iff $\Psi(A, \sigma) \models f$.

Consistency

D is **consistent** iff it has a model, otherwise it is **inconsistent**.

Queries of $\mathcal{A}$

Query

Any value statement of $\mathcal{A}$ is a *query*.

Consequence

Given an action domain D a query of $\mathcal{A}$ is a *consequence* of D iff it is true in every model of D .

Yale Shooting Problem (cont.)

Calculations

$\Sigma = \{\sigma_0, \sigma_1, \sigma_2, \sigma_3\}$ where

$$\sigma_0 = \{alive, \neg loaded\}$$

$$\sigma_1 = \{alive, loaded\}$$

$$\sigma_2 = \{\neg alive, \neg loaded\}$$

$$\sigma_3 = \{\neg alive, loaded\}.$$

$$\Psi(\text{LOAD}, \sigma_0) = \sigma_1$$

by (M1), (M2)

$$\Psi(\text{SHOOT}, \sigma_0) = \sigma_0$$

by (M1), (M2)

$$\Psi(\text{LOAD}, \sigma_1) = \sigma_1$$

by (M1), (M2)

$$\Psi(\text{SHOOT}, \sigma_1) = \sigma_2$$

by (M1)

$$\Psi(\text{LOAD}, \sigma_2) = \sigma_3$$

by (M1), (M2)

$$\Psi(\text{SHOOT}, \sigma_2) = \sigma_2$$

by (M1), (M2)

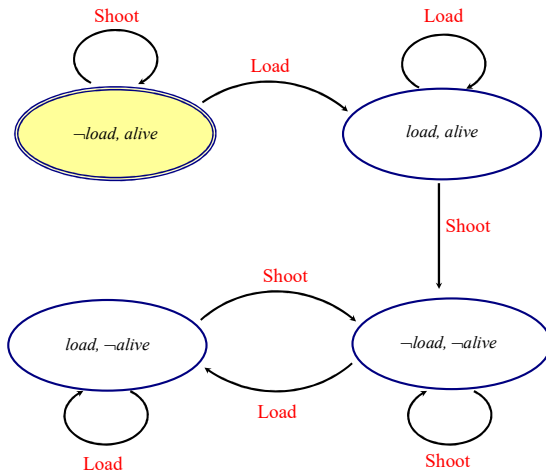
$$\Psi(\text{LOAD}, \sigma_3) = \sigma_3$$

by (M1), (M2)

$$\Psi(\text{SHOOT}, \sigma_3) = \sigma_2$$

by (M1), (M2).

Yale Shooting problem – Model



Example

Example 1.4

Consider the following action domain:

OPENTHEDOOR **causes** *open* **if** *hasCard*;

OPENTHEDOOR **causes** $\neg$ *open* **if** $\neg$ *hasCode*

Let $\sigma = \{hasCard, \neg hasCode\}$ and assume that OPENTHEDOOR is to be performed in σ . Due to the 1st statement, the resulting state satisfies *open*, but due to the 2nd statement it has to satisfy $\neg open$. Clearly, there is no state satisfying *open* and $\neg open$. Then there is no transition function Ψ defined for the action OPENTHEDOOR and the state σ , thus there is no structure $\mathcal{S}$ in which both statements are true. Hence that domain is **inconsistent**.

Thank you for your attention!

Questions are welcome.